Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 리스트의 개념과 동작 원리

리스트의 개념과 동작 원리 및 구현 방법을 설명할 수 있다.

02. 집합의 개념과 구현 방법

집합의 개념과 구현 방법을 설명할 수 있다.

리스트의
개념과 동작 원리

리스트의 개념

리스트(list), 선형 리스트(linear list)

순서를 가진 항목들의 모임

표현 방법	L = [$item_0$, $item_1$, $item_2$, ⋯, $item_(n-1)$]

참고

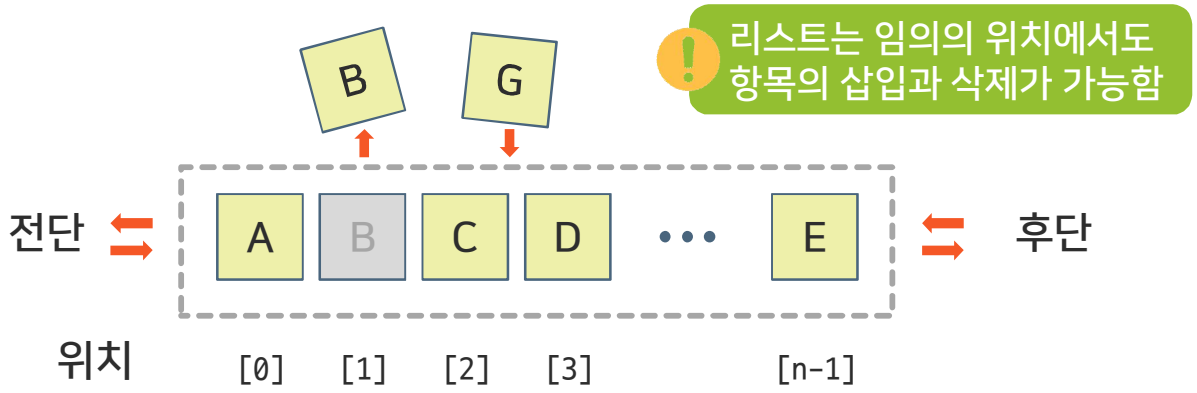
집합 ➡ 항목 간의 순서의 개념이 없음

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

리스트의 개념

리스트의 특징

1 항목들은 순서대로 나열되며, 각 항목은 자신의 위치가 있음



2 스택, 큐, 덱과는 자료의 접근 위치가 다름

리스트의 개념

리스트 ADT

데이터 같은 유형의 요소들의 순서 있는 모임

연산

- `List()`: 비어있는 새로운 리스트를 만듦
- `insert(pos, e)`: pos 위치에 새로운 요소 e를 삽입함
- `delete(pos)`: pos 위치에 있는 요소를 꺼내고(삭제) 반환함
- `isEmpty()`: 리스트가 비어있는지 검사함
- `getEntry(pos)`: pos 위치에 있는 요소를 반환함
- `size()`: 리스트 안의 요소의 개수를 반환함
- `clear()`: 리스트를 초기화함
- `find(item)`: 리스트에서 item이 있는지 찾아 인덱스를 반환함

리스트의 개념

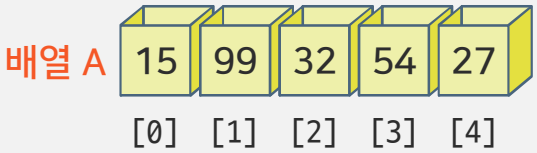
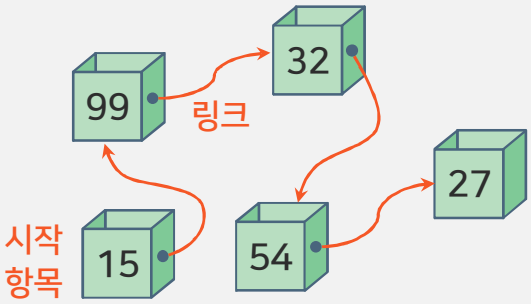
리스트 ADT

연산

- `replace(pos, item)`: pos에 있는 항목을 item으로 바꿈
- `sort()`: 리스트의 항목들을 어떤 기준으로 정렬함
- `merge(lst)`: 다른 리스트 lst를 리스트에 추가함
- `display()`: 리스트를 화면에 출력함
- `append(e)`: 리스트의 맨 뒤에 새로운 항목을 추가함

리스트의 개념

리스트 구현 방법

항목	배열 구조	연결된 구조
구현 형식	구현 간단	구현 복잡
항목 접근 복잡도	$O(1)$	$O(n)$
삽입 · 삭제	오버헤드 발생	효율적
항목 개수	개수 제한	크기 제한 없음
리스트		

리스트의 개념

리스트 용어 정리

파이썬 리스트	• C언어에서의 배열이 진화된 형태의 스마트한 배열 • 자료구조를 구현하기 위한 하나의 방법
연결 리스트	• 자료가 일렬로 나열된 연결된 구조 • 배열 구조에 대응
자료구조 리스트	• 추상적인 의미의 자료구조 리스트를 의미 • 리스트 ADT를 구현하기 위해 배열 구조나 연결 구조 사용

파이썬 리스트의 특징

파이썬 리스트

스마트한 배열

파이썬 리스트 선언	C 배열 선언
`A = [ 1, 2, 3, 4, 5 ]` `# 파이썬 리스트 A`	`int A[5] = { 1, 2, 3, 4, 5}` `// 정수 배열 A선언 및 초기화`

파이썬 리스트의 특징

파이썬 리스트

항목의 수

```
print('파이썬 리스트 A의 크기는', len(A))        # A의 크기(항목 수) 출력
```

항목 추가로 용량 증대 가능

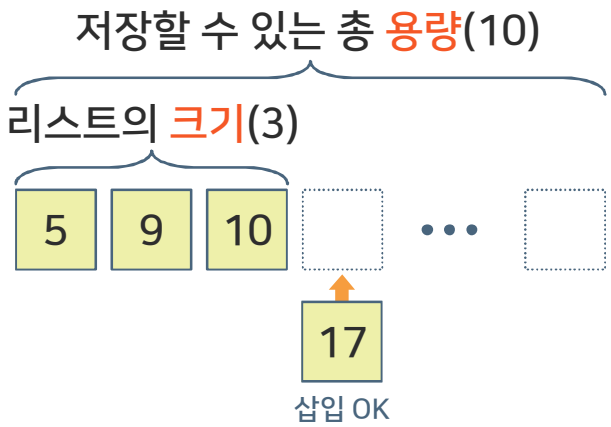
```
A.append(6)                    # A = [1, 2, 3, 4, 5, 6]
A.append(7)                    # A = [1, 2, 3, 4, 5, 6, 7]
A.insert(0, 0)                 # A = [0, 1, 2, 3, 4, 5, 6, 7]
```

파이썬 리스트의 특징

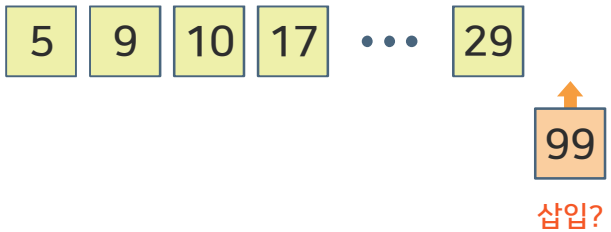
동적배열 특성

필요양보다 더 큰 메모리를 사용함

(크기 < 용량) 인 상황에서 항목 삽입



(크기 = 용량) 인 상황에서 항목 삽입



파이썬 리스트의 특징

동적배열 구조에서 용량 증가

Step 01

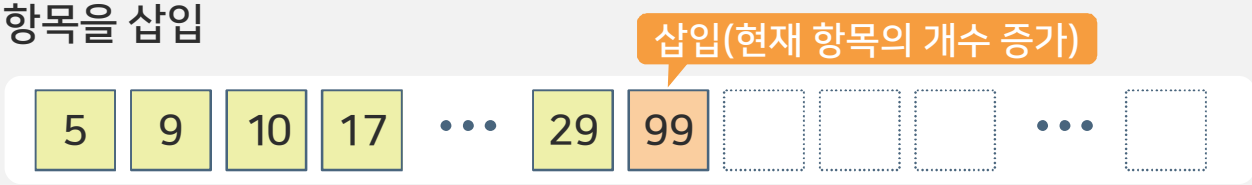
용량을 확장한 새로운 배열 할당 예 기존 배열 용량의 2배

Step 02

기존의 배열을 새로운 배열에 복사

Step 03

항목을 삽입



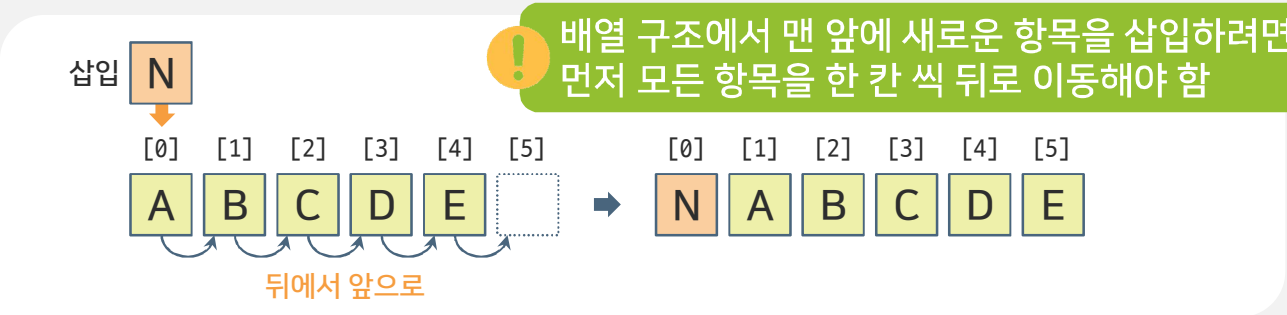
Step 04

기존 배열 해제, 리스트로 새 배열 사용

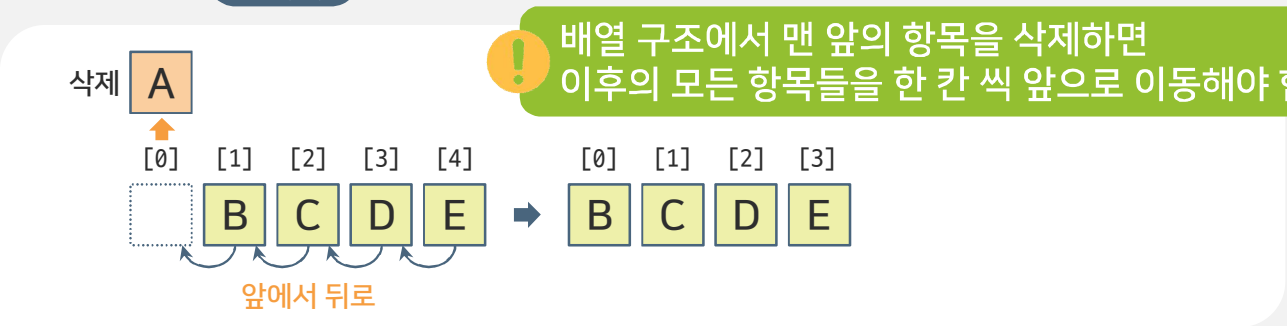
파이썬 리스트의 특징

리스트 연산의 시간 복잡도

- append(e) 대부분 O(1)
- insert(pos,e) O(n)



- pop(pos) O(n)



배열 리스트

함수를 이용한 배열 구현

자료구조 리스트의 ADT 구현

전역 변수와 함수로 구현

배열 리스트

함수를 이용한 배열 구현

```
items = []                                          # 리스트 항목 저장을 위한 파이썬 리스트
```

```
def insert(pos, elem) :                             # pos 위치에 새로운 요소 item을 삽입
   items.insert(pos, elem)                          # 파이썬 리스트 클래스의 insert 연산
def delete(pos) :                                   # pos 위치에 있는 요소를 꺼내고 반환
   return items.pop(pos)                            # 파이썬 리스트 클래스의 pop 연산
def getEntry(pos): return items[pos]                # pos번째 항목 반환
def isEmpty( ): return len(items) == 0              # 크기가 0이면 True 아니면 False
def size(): return len(items)                       # 리스트의 크기 반환, len()함수 이용
def clear(): items = []                             # items를 초기화 --> 오류
def find(item ): return items.index(item)           # 탐색 후 인덱스 반환
def replace(pos, elem): items[pos] = elem           # pos번째 항목 변경
def sort(): items.sort()                            # 정렬(sort 메소드 이용)
def merge(lst) : items.extend(lst)                  # 병합(확장)
def display(msg='ArrayList:'):                      # 출력: 디폴트 인수 사용
   print(msg, size(), items)                        # 메시지+크기+배열내용 출력
```

배열 리스트

함수를 이용한 배열 구현

테스트 프로그램

```
display( '파이썬 리스트로 구현한 리스트 테스트' )
insert(0, 10);        insert(0, 20);    insert(1, 30)
insert(size(),40);    insert(2, 50)
display( "파이썬 리스트로 구현한 List(삽입x5):" )
sort()
display( "파이썬 리스트로 구현한 List(정렬후):" )
replace(2, 90)
display( "파이썬 리스트로 구현한 List(교체x1):" )
```

```
delete(2);    delete(size()-1);    delete(0)
display( "파이썬 리스트로 구현한 List(삭제x3):" )
lst = [ 1, 2, 3 ]
merge(lst)
display( "파이썬 리스트로 구현한 List(병합+3):" )
clear()
display( "파이썬 리스트로 구현한 List(정리후):" )
```

C:\WINDOWS\system32\cmd.exe

```
파이썬 리스트로 구현한 리스트 테스트 0 []
파이썬 리스트로 구현한 List(삽입x5):  5 [20, 30, 50, 10, 40]
파이썬 리스트로 구현한 List(정렬후):  5 [10, 20, 30, 40, 50]
파이썬 리스트로 구현한 List(교체x1):  5 [10, 20, 90, 40, 50]
파이썬 리스트로 구현한 List(삭제x3):  2 [20, 40]
파이썬 리스트로 구현한 List(병합+3):  5 [20, 40, 1, 2, 3]
파이썬 리스트로 구현한 List(정리후):  5 [20, 40, 1, 2, 3]
```

clear() 연산 결과 오류

- 0 [ ]가 기대 출력
- items를 전역 변수로 선언해야 함
- global items 추가

```
def clear() :
    global items
    items = []
```

배열 리스트

클래스를 이용한 배열 구현

```
class ArrayList:
    def __init__( self ):                    # 생성자
        self.items = []                      # 클래스 변수 선언 및 초기화
```

```
def insert(self, pos, elem) :
    self.items.insert(pos, elem)
def delete(self, pos) :
    return self.items.pop(pos)
def isEmpty( self ):
    return self.size() == 0
def getEntry(self, pos) :
    return self.items[pos]
def size( self ):
    return len(self.items)
def clear( self ) :
    self.items = []
```

```
def find(self, item) :
    return self.items.index(item)
def replace(self, pos, elem) :
    self.items[pos] = elem
def sort(self) :
    self.items.sort()
def merge(self, lst) :
    self.items.extend(lst)
def display(self, msg='ArrayList:'):
    print(msg, '항목수=', self.size(), self.items)
```

배열 리스트

클래스를 이용한 배열 구현

테스트 프로그램

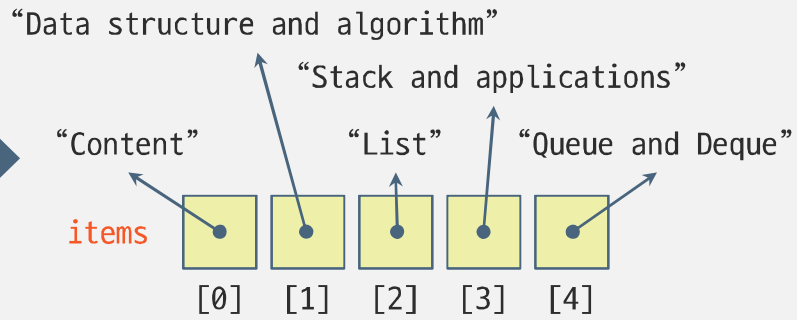
```
s = ArrayList()
s.display('파이썬 리스트로 구현한 리스트 테스트')
s.insert(0, 10);          s.insert(0, 20);     s.insert(1, 30)
s.insert(s.size(),40);    s.insert(2, 50)
s.display("파이썬 리스트로 구현한 List(삽입x5):")
s.sort()
s.display("파이썬 리스트로 구현한 List(정렬후):")
s.replace(2, 90)
s.display("파이썬 리스트로 구현한 List(교체x1):")
s.delete(2);     s.delete(s.size()-1);     s.delete(0)
s.display("파이썬 리스트로 구현한 List(삭제x3):")
lst = [ 1, 2, 3 ]
s.merge(lst)
s.display("파이썬 리스트로 구현한 List(병합+3):")
s.clear()
s.display("파이썬 리스트로 구현한 List(정리후):")
```

리스트 응용

라인 편집기 기능 정의

i	라인 삽입	행 번호와 문자열을 입력하면 그 행에 한 라인 추가
d	한 라인 삭제	행 번호를 입력하면 그 행을 삭제
r	한 라인 변경	행 번호와 문자열을 입력하면 그 행의 내용을 변경
p	현재 내용 출력	현재 문서의 모든 내용을 라인 번호와 함께 출력
l	파일 입력	지정된 (test.txt) 파일로부터 라인을 읽어 들임
s	파일 출력	지정된 (test.txt) 파일로 편집 내용을 저장

```
Contents
Data structure and algorithm
List
Stack and applications
Queue and Deque
```



리스트 응용

라인 편집기 코드

```
def myLineEditor() :                                    # 라인 편집기 주 함수
    list = ArrayList()                                  # 리스트 객체 생성
    while True :
        command = input( "[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=>" )

        if command == 'i' :                             # 삽입 연산
            pos = int( input( " 입력행 번호: " ))       # 삽입할 행 번호 입력
            str = input( " 입력행 내용: " )             # 삽입할 행 내용 입력
            list.insert(pos, str)                       # insert 메소드로 삽입
        elif command == 'd' :                           # 행 삭제
            pos = int( input( " 삭제행 번호: " ))       # 삭제할 행 번호 입력
            list.delete(pos)                            # delete 메소드로 삭제
        elif command == 'r' :                           # 행 내용 변경
            pos = int( input( " 변경행 번호: " ))       # 변경할 행 번호 입력
            str = input( " 변경행 내용: " )             # 변경할 행 내용 입력
            list.replace(pos, str)
                    ⋮                           # replace로 변경
        elif command == 'q' : return
```

리스트 응용

라인 편집기 코드

실행결과 1

C:\WINDOWS\system32\cmd.exe

```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> i
  입력행 번호: 0
  입력행 내용: Contents
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> i
  입력행 번호: 1
  입력행 내용: 자료구조와 알고리즘
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> i
  입력행 번호: 2
  입력행 내용: 리스트
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> i
  입력행 번호: 3
  입력행 내용: 스택, 큐, 덱
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] Contents
[ 1] 자료구조와 알고리즘
[ 2] 리스트
[ 3] 스택, 큐, 덱
```

0번 줄에 삽입

1번 줄에 삽입

2번 줄에 삽입

3번 줄에 삽입

라인 추가 명령

화면 출력 명령

내용 출력

(1/2)

리스트 응용

라인 편집기 코드

실행결과 1

C:\WINDOWS\system32\cmd.exe

삭제 명령

```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> d
삭제행 번호: 1
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] Contents
[ 1] 리스트
[ 2] 스택, 큐, 덱

[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=>
```

1번 줄 삭제

(2/2)

리스트 응용

라인 편집기 코드

실행결과 2

C:\WINDOWS\system32\cmd.exe

Text.txt를 읽음

```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> 1
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] class ArrayList:
[ 1]     def __init__( self ):
[ 2]         self.items = []
[ 3]
[ 4]     def insert(self, pos, elem) : self.items.insert(pos, elem)
[ 5]     def delete(self, pos) : self.items.pop(pos)
[ 6]     def isEmpty( self ) : return self.size() == 0
```

내용 출력

(1/2)

리스트 응용

라인 편집기 코드

실행결과 2

C:\WINDOWS\system32\cmd.exe

변경 명령

```
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> r
  변경행 번호: 5
  변경행 내용:     def delete(this, pos): this.items.pop(pos)
[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=> p
Line Editor
[ 0] class ArrayList:
[ 1]     def __init__( self ):
[ 2]         self.items = []
[ 3]
[ 4]     def insert(self, pos, elem) : self.items.insert(pos, elem)
[ 5]     def delete(this, pos): this.items.pop(pos)
[ 6]     def isEmpty( self ) : return self.size() == 0

[메뉴선택] i-입력, d-삭제, r-변경, p-출력, 1-파일읽기, s-저장, q-종료=>
```

self를 this로 수정

(2/2)

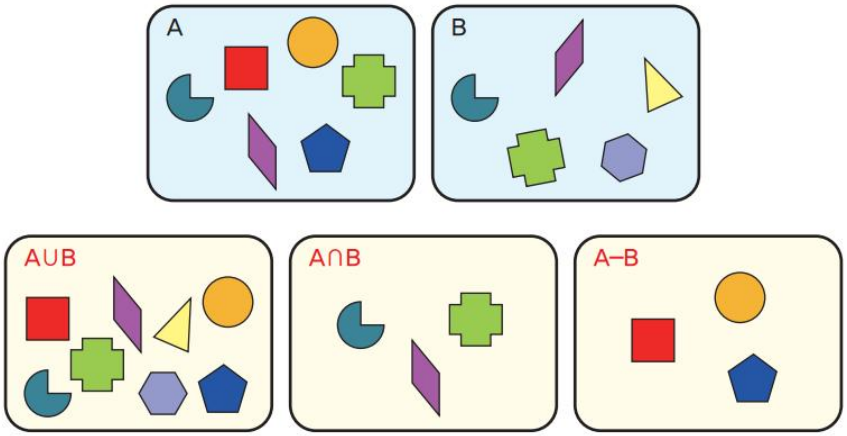
집합의
개념과 구현 방법

집합의 개념

집합의 특징

1. 원소의 중복을 허용하지 않음
2. 원소들 사이에 순서가 없음 → 선형 자료구조가 아님

표현 방법 $S = \{ item_0, item_1, item_2, \cdots, item_(n-1) \}$



집합의 개념

집합 ADT

데이터

- 같은 유형의 유일한 요소들의 모임
- 원소들은 순서는 없지만 서로 비교할 수 있어야 함

연산

- `set()`: 비어있는 새로운 집합을 만듦
- `size()`: 집합의 원소의 개수를 반환함
- `contains(e)`: 집합이 원소 e를 포함하는지 검사하고 반환함
- `insert(e)`:
  새로운 원소 e를 삽입하고, 이미 e가 있다면 삽입하지 않음
- `delete(e)`: 원소 e를 집합에서 꺼내고(삭제) 반환함

집합의 개념

집합 ADT

연산

- `equals(setB)`: setB와 같은 집합인지 검사함
- `union(setB)`: setB와의 합집합을 만들어 반환함
- `intersect(setB)`: setB와의 교집합을 만들어 반환함
- `difference(setB)`: setB와의 차집합을 만들어 반환함
- `display()`: 집합을 화면에 출력함

집합의 구현

리스트를 이용한 집합의 구현

```
class Set:                          # 집합 클래스
    def __init__(self):             # 생성자
        self.items = []             # 원소를 저장하기 위한 리스트 생성

    def size(self):                 # 집합의 크기
        return len(self.items)      #  len() 함수 사용
    def display(self, msg)          # 화면에 출력
        print(msg, self.items)      # 메시지 + 집합 내용 출력
```

집합의 구현

리스트를 이용한 집합의 구현

```
def contains(self, item) :
    return item in self.items
```

```
def contains(self, item) :
    for i in range(len(self.items)):
        if self.items[i] == item :
            return True
    return False
```

```
def insert(self, elem) :
    if elem not in self.items :
        self.items.append(elem)
```

```
def delete(self, elem) :
    if elem in self.items :
        self.items.remove(elem)
```

집합의 구현

리스트를 이용한 집합의 구현

```
def union( self, setB ):
    setC = Set()
    setC.items = list(self.items)
    for elem in setB.items :
        if elem not in self.items :
            setC.items.append(elem)
    return setC
```

```
def intersect( self, setB ):
    setC = Set()
    for elem in self.items :
        if elem in setB.items :
            setC.items.append(elelm)
    return setC
```

```
def difference( self, setB ):
    setC = Set()
    for elem in self.items:
        if elem not in setB.items:
            setC.items.append(elem)
    return setC
```

집합의 구현

테스트 코드

01

```
setA = Set()
setA.insert('휴대폰')
setA.insert('지갑')
setA.insert('손수건')
setA.display('Set A:')
```

```
setB = Set()
setB.insert('빗')
setB.insert('파이썬 자료구조')
setB.insert('야구공')
setB.insert('지갑')
setB.display('Set B:')
```

C:\WINDOWS\system32\cmd.exe

```
Set A: ['휴대폰', '지갑', '손수건']
Set B: ['빗', '파이썬 자료구조', '야구공', '지갑']
```

집합의 구현

테스트 코드

02

```
setB.insert('빗')
setA.delete('손수건')
setA.delete('발수건')
setA.display('Set A:')
setB.display('Set B:')
```

C:\WINDOWS\system32\cmd.exe

```
Set A: [ '휴대폰' , '지갑' , '손수건' ]
Set B: [ '빗' , '파이썬 자료구조' , '야구공' , '지갑' ]
Set A: [ '휴대폰' , '지갑' ]
Set B: [ '빗' , '파이썬 자료구조' , '야구공' , '지갑' ]
```

'빗'을 중복해서 넣었지만
하나만 들어있음

집합의 구현

테스트 코드

03

```
setA.union(setB).display('A U B:')
setA.intersect(setB).display('A ^ B:')
setA.difference(setB).display('A - B:')
```

C:\WINDOWS\system32\cmd.exe

```
Set A: [ '휴대폰' , '지갑' ]
Set B: [ '빗' , '파이썬 자료구조' , '야구공' , '지갑' ]
A U B: [ '휴대폰' , ' 지갑' , '빗' , '파이썬 자료구조' , '야구공' ]
A ^ B: [ '지갑' ]
A - B: [ '휴대폰' ]
```

합집합

교집합

차집합

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
1	리스트에 대한 설명 중 옳지 않은 것은?	① 원소의 삽입과 삭제 위치가 전단으로 제한된다. ② 선형 자료구조이다. ③ 배열 구조와 연결된 구조로 구현할 수 있다. ④ 원소들의 중복을 허용한다.	
2	파이썬 리스트의 특징으로 옳지 않은 것은?	① 파이썬 리스트는 "배열 구조"로 구현 ② 음수 인덱스는 허용 불가 ③ 인덱스 연산자를 이용해 항목에 접근 가능 ④ 인덱스는 0번부터 시작	
3	배열 구조로 구현된 리스트의 특징으로 바르지 않은 것은?	① 구현이 비교적 간단하고 항목 접근이 효율적 ② 삽입 삭제 시 오버헤드가 발생 ③ 삽입 삭제가 효율적 ④ 항목의 개수가 제한될 수 있음	
4	파이썬 리스트의 append() 연산에 대한 시간 복잡도로 가장 옳은 것은?	① 항상 $O(1)$ ② 항상 $O(n)$ ③ 대부분의 경우 $O(n)$ ④ 대부분의 경우 $O(1)$	

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	

5	공백 상태의 리스트에서, 다음 연산을 차례로 수행한 후에 리스트의 내용으로 옳은 것은?
	`insert(0,10);`
	`insert(1,20);`
	`insert(0,30);`
	`insert(2,40);`
	`insert(size(),50);`
		① [30, 10, 40, 20, 50]
		② [10, 20, 30, 40, 50]
		③ [50, 40, 30, 20, 10]
		④ [30, 10, 20, 40, 50]	

6	집합의 특징으로 옳지 않은 것은?	① 원소의 중복을 불허 ② 원소의 중복을 허용 ③ 원소들 사이에 순서가 없음 ④ 집합의 표현은 {}에 나열	
7	집합 ADT에서 '데이터'는 동일 유형의 유일한 요소들의 모임으로. 순서는 없지만 서로 비교할 수 없어야 한다.	① O ② X	

학습정리

리스트와 집합

리스트의 개념과 동작 원리

- 리스트는 항목들이 차례로 나열된 선형 자료구조로 순서를 가진 항목들의 모임이다.
- 배열 구조의 리스트는 구현이 간단하고 항목 접근이 효율적이나, 삽입 삭제 시 오버헤드가 발생한다.
- 연결된 구조의 리스트는 구현이 복잡하지만, 삽입 삭제가 효율적이고 크기가 제한되지 않는다.
- 파이썬 리스트는 배열구조로 구현하며 인덱스는 0부터 시작하고 음수 인덱스도 가능하다.
- 리스트 연산의 복잡도
  - append(e) 연산: 대부분 O(1)
  - insert(pos, e) 연산: O(n)
  - pop(pos) 연산 : O(n)

집합의 개념과 구현 방법

- 집합은 원소의 중복을 허용하지 아니한다.
- 집합의 원소들 사이에는 순서가 없고 선형 구조가 아니다.
- 데이터는 동일 유형의 유일한 요소들의 모임으로 서로 비교가능해야 한다.